

SysPilot Master Architecture Specification

Complete 4-Volume Technical Reference Manual for Production
Linux Observability

Kernel eBPF Probes · Edge Daemon Pipeline · Columnar TSDB · Causal Graph Engine

Document Ref: SIOP-MASTER-BOOK-2026

Status: Complete Systems Design & Class Catalog

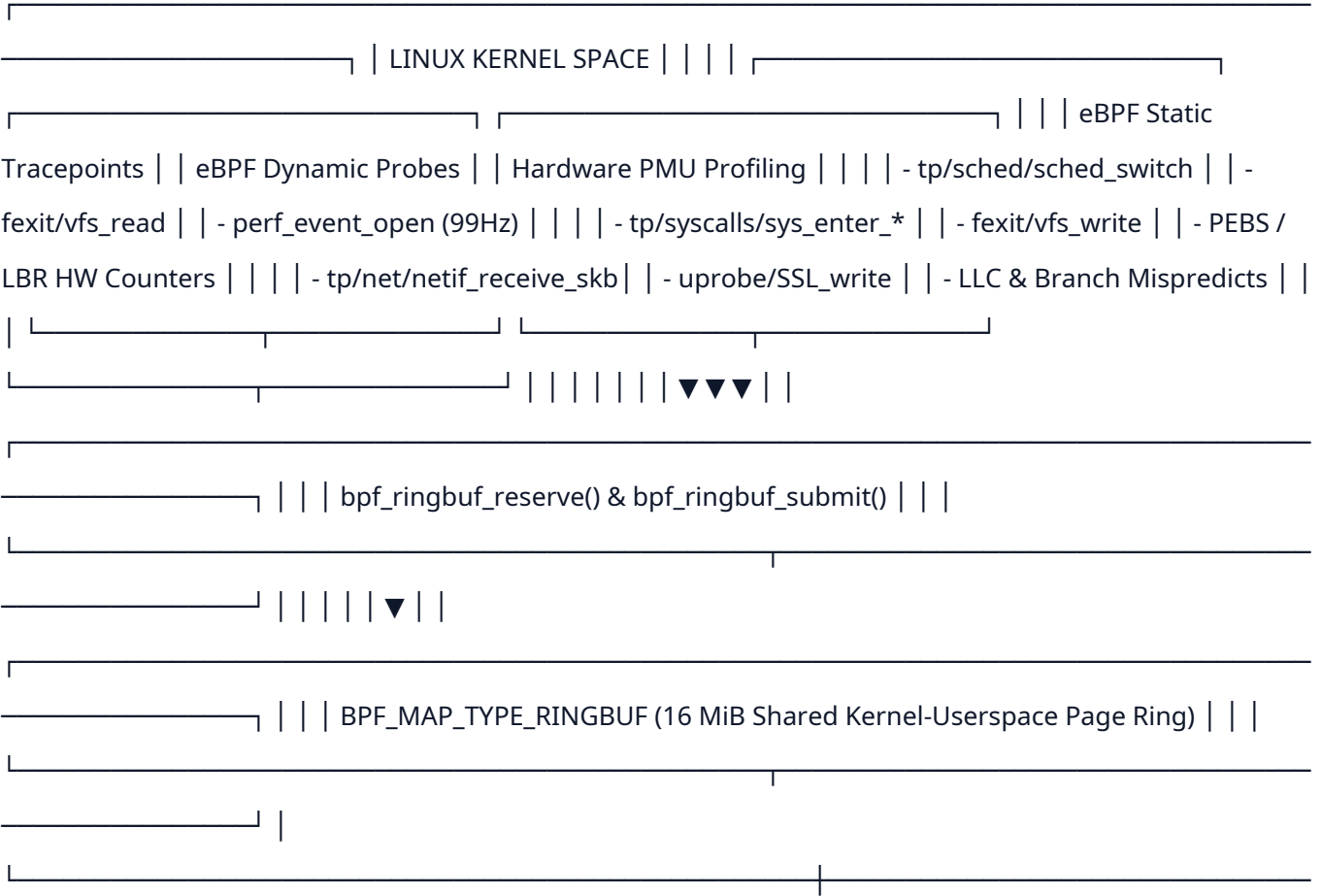
SysPilot Master Architecture & Systems Specification Manual

SysPilot Master Architecture Specification: Volume 1

Kernel Telemetry, eBPF CO-RE Probes & Subsystem Collector Mechanics

1. Executive Kernel Subsystem Architecture

The kernel telemetry layer of the **SysPilot Infrastructure Observability Platform (SIOP)** provides zero-patch, event-driven visibility into Linux system operations. Operating entirely within kernel space via Extended Berkeley Packet Filter (eBPF) bytecode, hardware Performance Monitoring Units (PMUs), and kernel multicast sockets, SIOP captures process execution, thread scheduling, memory allocations, VFS file I/O, network socket traffic, and hardware counters with nanosecond precision.



_____ | epoll_wait() / zero-copy mmap read ▼

_____ | USERSPACE DAEMON (syspilotd) |

_____ | ... ---

2. eBPF CO-RE Probe Specifications & Attach Points

2.1 Static Tracepoint Attachment Matrix

Static kernel tracepoints present stable, version-independent ABI contracts. They carry zero performance overhead when probes are disabled.

Subsystem	Tracepoint Attach Path	Trigger Event	Primary Telemetry Extracted
Scheduler	<code>`tp/sched/sched_switch`</code>	Task context switch	Prev/Next PID, TID, state, execution CPU, run-queue latency
Syscall Entry	<code>`tp/syscalls/sys_enter_*`</code>	System call invocation	Syscall ID, input pointers, process credentials (<code>`uid`</code> , <code>`gid`</code>)
Syscall Exit	<code>`tp/syscalls/sys_exit_*`</code>	System call return	Syscall ID, return value (<code>`int64_t`</code>), execution latency
Networking	<code>`tp/net/netif_receive_skb`</code>	Packet ingress	Socket buffer length, protocol, interface index, cgroup ID
Block I/O	<code>`tp/block/block_rq_issue`</code>	Disk request dispatch	Device major/minor, sector size, write/read operation bitmask

2.2 Dynamic Probe & Trampoline Attach Specifications (``fexit`` / **Kprobes**)

For internal kernel functions lacking static tracepoints, SIOP uses eBPF **``fexit`` trampolines** (Linux 5.5+), which replace traditional ``kretprobes``. ``fexit`` probes execute inline via direct kernel function trampolines, avoiding the breakpoint interrupt trap cost of legacy kprobes.

```
```c // BPF program attached to VFS file read entry and exit via fexit trampoline
SEC("fexit/vfs_read") int BPF_PROG(trace_vfs_read_exit, struct file *file,
char __user *buf, size_t count, loff_t *pos, ssize_t ret) {
 uint64_t pid_tgid = bpf_get_current_pid_tgid();
 uint32_t pid = pid_tgid >> 32;
 uint32_t tid = (uint32_t)pid_tgid;
 // Filter check: fast out if PID is ignored if (is_pid_filtered(pid)) return 0;
 // Reserve space inside kernel-userspace ring buffer
 struct bpftrace_event_t *ev;
 ev = bpf_ringbuf_reserve(&events_ringbuf, sizeof(*ev), 0);
 if (!ev) { // Increment atomic drop counter
 map_on_ring_buffer_full_increment_drop_counter(EVENT_TYPE_VFS_READ);
 return 0;
 }
 // Populate event fields directly in ring memory (zero-copy)
 ev->timestamp_ns = bpf_ktime_get_ns();
 ev->pid = pid;
 ev->tid = tid;
 ev->cgroup_id = bpf_get_current_cgroup_id();
 ev->event_type = EVENT_TYPE_VFS_READ;
 ev->ret_val = ret;
 ev->arg1 = count;
 ev->arg2 = (uint64_t)file;
 bpf_ringbuf_submit(ev, 0);
 return 0;
}
```
```

3. Kernel Memory Layouts & Data Structure Contracts

3.1 64-Byte Cache-Aligned Kernel Event Struct (`bpftrace\_event\_t`)

```
``c // Explicitly packed and 64-byte aligned to match CPU cache lines and avoid false sharing struct
__attribute__((packed, aligned(64))) bpftrace_event_t { uint64_t timestamp_ns; // Kernel monotonic time (ns)
via bpf_ktime_get_ns() uint32_t pid; // Process ID (TGID in kernel terms) uint32_t tid; // Thread ID (PID in
kernel terms) uint32_t ppid; // Parent Process ID uint32_t uid; // User ID of process context uint64_t
cgroup_id; // Kernel cgroup v2 inode ID uint16_t event_type; // Telemetry event code identifier uint8_t
cpu_id; // Hardware CPU core ID uint8_t flags; // Bit 0: Dropped; Bit 1: Anomaly; Bit 2: Sampled int64_t
ret_val; // Syscall return status / byte count uint64_t arg1; // Generic payload slot 1 (FD, pointer, bytes)
uint64_t arg2; // Generic payload slot 2 (address, port, flags) uint64_t stack_id; // eBPF stack trace map
index }; ``
```

3.2 eBPF Map Definitions

```
``c // 1. Primary Kernel-Userspace Ring Buffer Map (16 MiB pre-allocated) struct { __uint(type,
BPF_MAP_TYPE_RINGBUF); __uint(max_entries, 16 * 1024 * 1024); } events_ringbuf SEC(".maps"); // 2. Kernel
Stack Trace Storage Map (32,768 unique stack call chains) struct { __uint(type, BPF_MAP_TYPE_STACK_TRACE);
__uint(max_entries, 32768); __uint(key_size, sizeof(uint32_t)); __uint(value_size, 128 * sizeof(uint64_t)); // Up to
128 instruction pointers per stack } stack_traces SEC(".maps"); // 3. Per-CPU Event Loss & Drop Counters
struct { __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY); __uint(max_entries, 64); __type(key, uint32_t);
__type(value, uint64_t); } drop_counters SEC(".maps"); `` ---
```

4. Hardware PMU & Process Connector Subsystems

4.1 PMU Profiler (`perf\_event\_open` @ 99Hz)

SIOP configures PMU hardware counters to sample instruction addresses and hardware metrics at 99 Hz to prevent harmonic synchronization with 100 Hz timer interrupts. ``c struct perf_event_attr pe_attr = { .type = PERF_TYPE_HARDWARE, .size = sizeof(struct perf_event_attr), .config = PERF_COUNT_HW_INSTRUCTIONS, .sample_freq = 99, // 99 Hz sampling frequency .sample_type = PERF_SAMPLE_IP | PERF_SAMPLE_TID | PERF_SAMPLE_CALLCHAIN | PERF_SAMPLE_CPU, .freq = 1, .disabled = 1, .exclude_kernel = 0, // Profile both user and kernel space .precise_ip = 2, // Enable PEBS (Precise Event-Based Sampling) on Intel }; ``

4.2 Netlink Process Lifecycle Listener (`NETLINK\_CONNECTOR`)

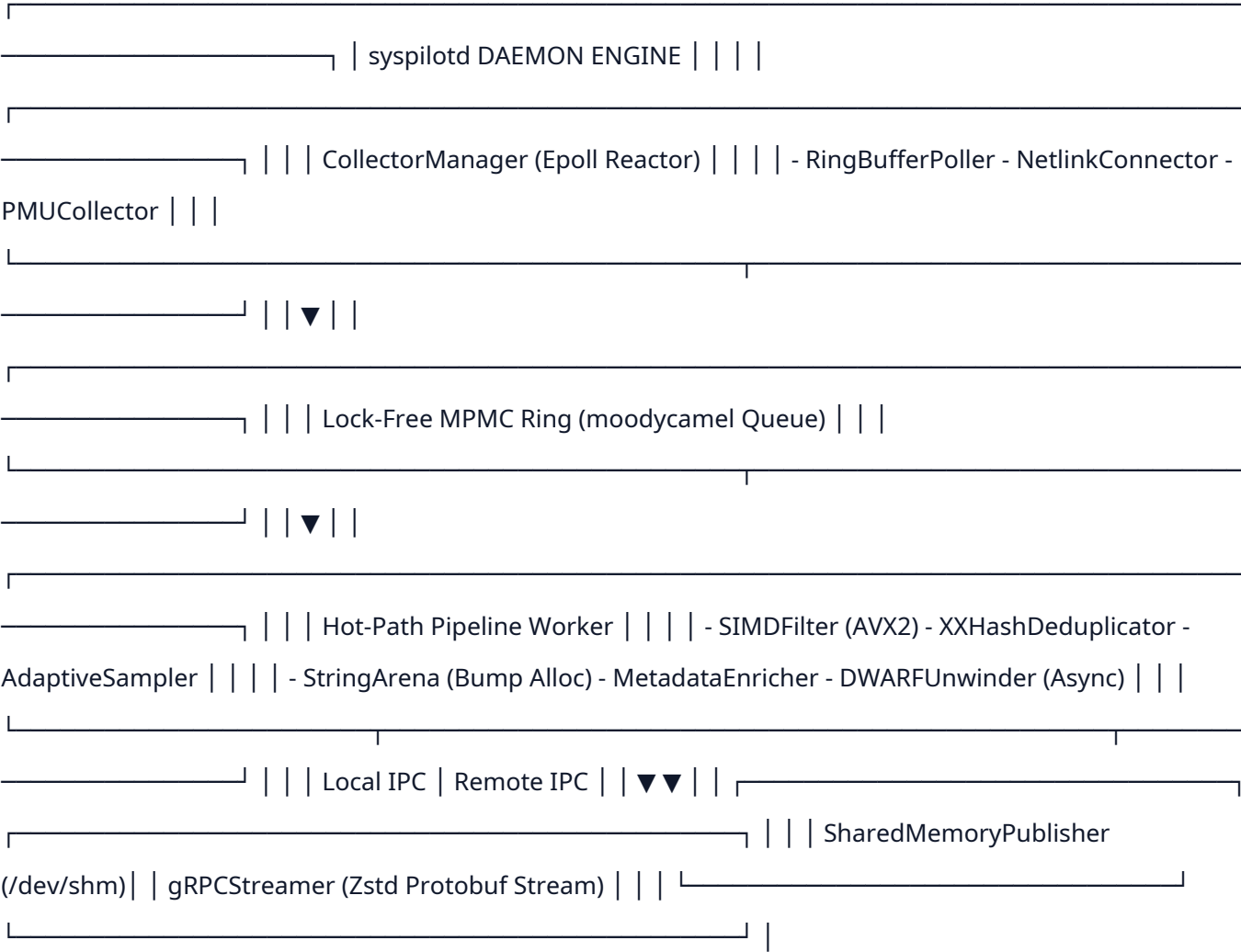
Listens to `PROC\_EVENT\_FORK`, `PROC\_EVENT\_EXEC`, `PROC\_EVENT\_EXIT`, and `PROC\_EVENT\_UID` events over a kernel multicast Netlink socket (`CN\_IDX\_PROC`), achieving zero-polling process execution tracing.

SysPilot Master Architecture Specification: Volume 2

Edge Daemon Architecture, Processing Pipeline & C++ Class Catalog

1. Executive Daemon Subsystem Architecture

The edge daemon (`syspilotd`) is a high-performance C++20 userspace process responsible for ingesting raw eBPF ring buffer byte streams, filtering and deduplicating events using SIMD instructions, enriching records with cgroup/container metadata, and publishing processed telemetry to local and central targets. ``



2. Complete C++ Class Catalog & Interface Specifications

2.1 Collection Layer Interfaces & Classes

```
##### `ICollector` Interface (`src/collector/icollector.hpp`) ```cpp namespace syspilot::collector { class
ICollector { public: virtual ~ICollector() = default; // Initializes collector hardware/kernel resources virtual bool
start() = 0; // Stops collection loop and unbinds probes/sockets virtual void stop() = 0; // Returns file
descriptor for epoll integration (-1 if poll-free) virtual int get_fd() const noexcept = 0; // Human-readable
collector identifier virtual std::string_view name() const noexcept = 0; }; } // namespace syspilot::collector ```
##### `RingBufferPoller` Class (`src/collector/ring_buffer_poller.hpp`) ```cpp namespace syspilot::collector
{ class alignas(64) RingBufferPoller final : public ICollector { private: int ring_fd_{-1}; struct ring_buffer*
rb_{nullptr}; moodycamel::ConcurrentQueue* target_queue_{nullptr}; std::atomic events_polled_{0};
std::atomic events_dropped_{0}; public: explicit RingBufferPoller(moodycamel::ConcurrentQueue* queue);
~RingBufferPoller() override; bool start() override; void stop() override; int get_fd() const noexcept override
{ return ring_fd_; } std::string_view name() const noexcept override { return "RingBufferPoller"; } // Polls eBPF
ring buffer pages and enqueues records (lockless) int poll(int timeout_ms) noexcept; uint64_t total_polled()
const noexcept { return events_polled_.load(std::memory_order_relaxed); }; }; } // namespace
syspilot::collector ```
##### `NetlinkConnector` Class (`src/collector/netlink_connector.hpp`) ```cpp namespace
syspilot::collector { class NetlinkConnector final : public ICollector { private: int nl_fd_{-1}; struct sockaddr_nl
local_addr_; alignas(64) char recv_buf_[8192]; public: NetlinkConnector(); ~NetlinkConnector() override;
bool start() override; void stop() override; int get_fd() const noexcept override { return nl_fd_; }
std::string_view name() const noexcept override { return "NetlinkConnector"; } // Processes incoming process
fork/exec/exit events from kernel socket void handle_netlink_event() noexcept; }; } // namespace
syspilot::collector ``` ---
```

2.2 Processing Pipeline Classes

```
##### `StringArena` Bump Allocator (`src/memory/string_arena.hpp`) ```cpp namespace syspilot::memory
{ class StringArena { private: static constexpr size_t CHUNK_SIZE = 256 * 1024; // 256 KiB Slabs struct Chunk
{ alignas(64) char data[CHUNK_SIZE]; }; std::vector< Chunk > chunks_; size_t current_chunk_{0}; size_t
current_offset_{0}; public: StringArena(); ~StringArena() = default; // Fast bump-allocation for strings (O(1)
amortized, zero malloc) std::string_view allocate(std::string_view str); // Recycles all memory chunks in O(1)
```



```

time void reset() noexcept; size_t total_allocated_bytes() const noexcept; }; } // namespace syspilot::memory
```
`SIMDFilter` Vector Engine (`src/pipeline/simd_filter.hpp`) ```cpp namespace syspilot::pipeline { struct
alignas(64) EventBatch { static constexpr size_t BATCH_CAPACITY = 1024; bpftrace_event_t
events[BATCH_CAPACITY]; size_t count{0}; }; class SIMDFilter { private: uint32_t target_pid_filter_{0}; public:
explicit SIMDFilter(uint32_t target_pid = 0) : target_pid_filter_(target_pid) {} // SIMD AVX2 vectorized PID
filtering across 8 events per iteration size_t filter_pids_avx2(EventBatch& batch, uint32_t target_pid)
noexcept; // Fallback scalar filter for non-AVX2 CPUs size_t filter_pids_scalar(EventBatch& batch, uint32_t
target_pid) noexcept; }; } // namespace syspilot::pipeline ```
`AdaptiveSampler` Load Shedding Class
(`src/pipeline/adaptive_sampler.hpp`) ```cpp namespace syspilot::pipeline { class AdaptiveSampler { private:
double sample_rate_{1.0}; uint64_t total_seen_{0}; uint64_t total_sampled_{0}; public: AdaptiveSampler() =
default; // Evaluates event sampling status based on host CPU usage and event flags bool
should_sample(const bpftrace_event_t& ev, double host_cpu_pct) noexcept; double current_sample_rate()
const noexcept { return sample_rate_; } }; } // namespace syspilot::pipeline ``` ---

```

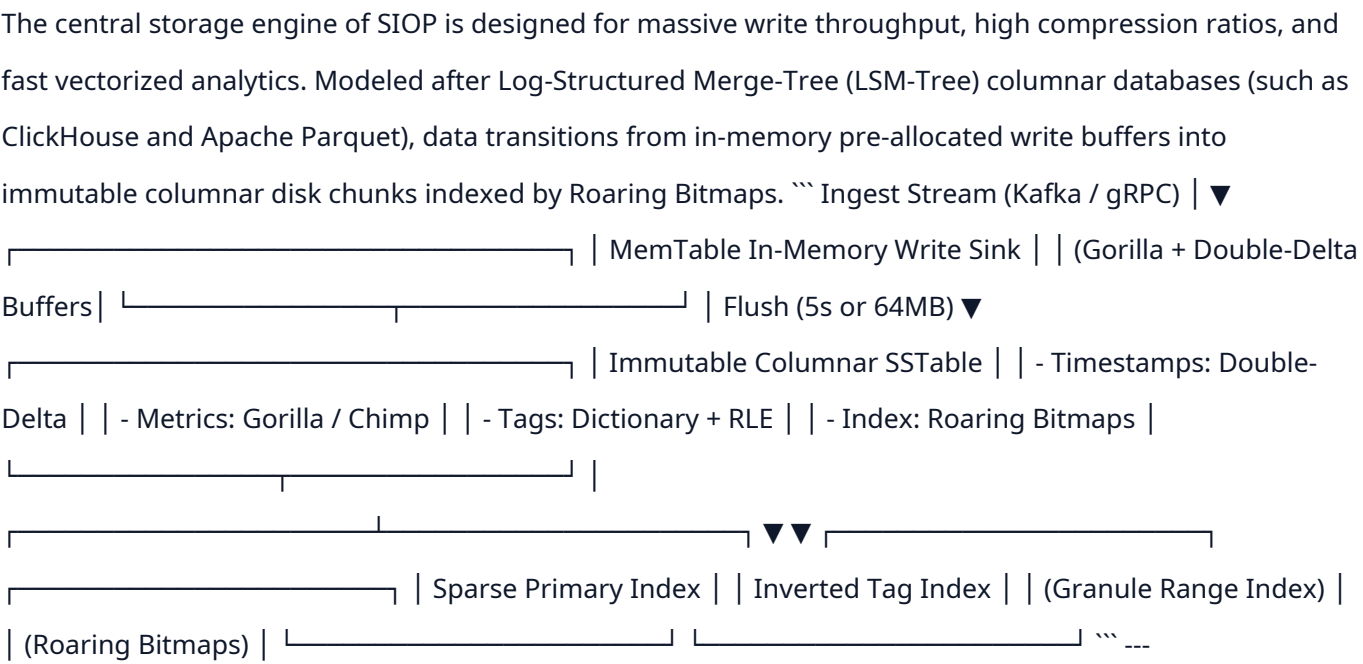
## 2.3 Memory Alignment & Threading Models

Class Name	Threading Model	Memory Alignment	Primary Allocation Strategy
`RingBufferPoller`	Single-Producer Thread	64-byte Cache Aligned	Stack buffers + Mmap ring
`CollectorManager`	Main Reactor Loop (`epoll`)	Standard	Member vector initialization
	Thread-Local / Single Worker	64-byte Cache Aligned	Pre-allocated 256 KiB Chunk Slabs
	Worker Pipeline Threads	64-byte Cache Aligned	In-place register mutation
`SharedMemoryPublisher`	Non-blocking Lockless	4096-byte Page Aligned	Memory mapped file (`/dev/shm`)

# SysPilot Master Architecture Specification: Volume 3

## Columnar TSDB Storage Engine, Encoding Algorithms & Inverted Indexing

### 1. Executive Storage Engine Architecture



### 2. Columnar Encoding Specifications

#### 2.1 Gorilla Floating-Point Encoding Engine (src/storage/gorilla\_encoder.hpp)

The Gorilla compression algorithm compresses 64-bit floating-point metrics (such as CPU utilization percentages and I/O rates) by calculating XOR differences between consecutive metric values.

```
namespace syspilot::storage {
class GorillaEncoder {
private:
 uint64_t last_val_bits{0};
 uint32_t last_leading_zeros{0xFFFFFFFF};
 uint32_t last_trailing_zeros{0};
 std::vector buffer_;
 size_t bit_offset{0};
public:
 GorillaEncoder() {
 buffer_.reserve(64 * 1024); // Reserve 64 KiB buffer
 }
 void encode_double(double
```

```

value) { uint64_t val_bits; std::memcpy(&val_bits, &value, sizeof(double)); if (buffer_.empty()) { // Write first
value uncompressed (64 bits) append_bits(val_bits, 64); last_val_bits_ = val_bits; return; } uint64_t xor_val =
val_bits ^ last_val_bits_; if (xor_val == 0) { // Control bit 0: Value identical to previous value append_bit(0); }
else { // Control bit 1: Value differs append_bit(1); uint32_t leading = __builtin_clzll(xor_val); uint32_t trailing =
__builtin_ctzll(xor_val); if (leading >= last_leading_zeros_ && trailing >= last_trailing_zeros_) { // Control bit 0:
Reuse previous leading/trailing zero boundaries append_bit(0); uint32_t bits_to_write = 64 -
last_leading_zeros_ - last_trailing_zeros_; append_bits(xor_val >> last_trailing_zeros_, bits_to_write); } else { //
Control bit 1: Write new leading and length bounds append_bit(1); last_leading_zeros_ = leading;
last_trailing_zeros_ = trailing; append_bits(leading, 5); // 5 bits for leading zero count (0-31) uint32_t length =
64 - leading - trailing; append_bits(length, 6); // 6 bits for length (0-63) append_bits(xor_val >> trailing,
length); } } last_val_bits_ = val_bits; } private: void append_bit(uint8_t bit) { // Bitwise packing
implementation } void append_bits(uint64_t val, uint8_t num_bits) { // Bitwise packing implementation } }; //
namespace syspilot::storage ``` ---

```

### 3. Inverted Indexing via Roaring Bitmaps

SIOP maintains multi-dimensional tag search capabilities (e.g. querying across specific container IDs, host names, or error exit codes) using compressed **Roaring Bitmaps**. ```cpp namespace syspilot::storage

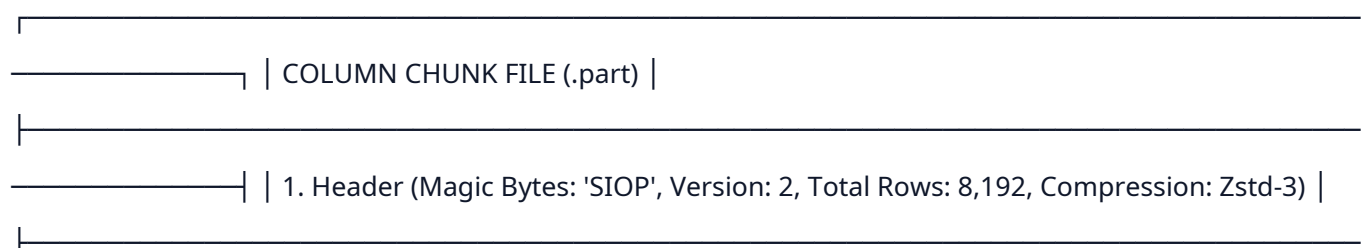
```

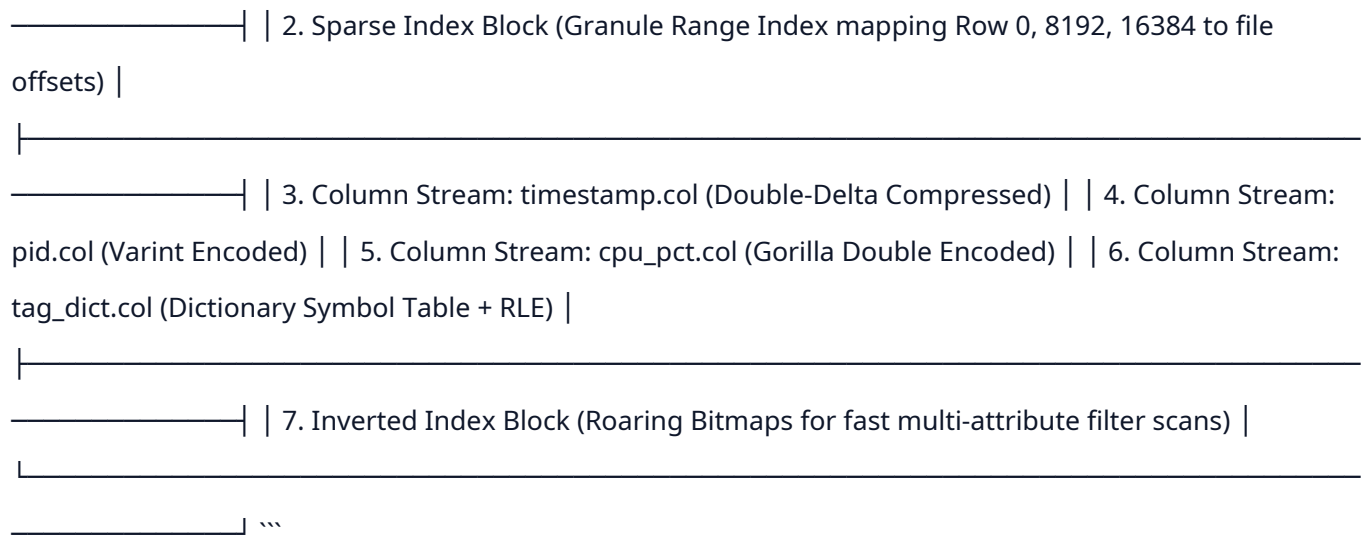
{ class RoaringBitmapIndex { private: // Map tag string -> bitmap of matching row IDs tsl::robin_map index_;
public: void insert(const std::string& tag, uint32_t row_id) { index_[tag].add(row_id); } // Fast set operations
across multiple criteria roaring::Roaring query_and(const std::string& tag1, const std::string& tag2) const
{ auto it1 = index_.find(tag1); auto it2 = index_.find(tag2); if (it1 == index_.end() || it2 == index_.end()) { return
roaring::Roaring(); } roaring::Roaring result = it1->second; result &= it2->second; // Fast SIMD bitmap AND
operation return result; } }; // namespace syspilot::storage ``` ---

```

### 4. Columnar Disk Layout & Part Compaction

Columnar disk chunks are organized into immutable files containing block header metadata, primary key range sparse indexes, encoded column streams, and bitmap indexes: ```





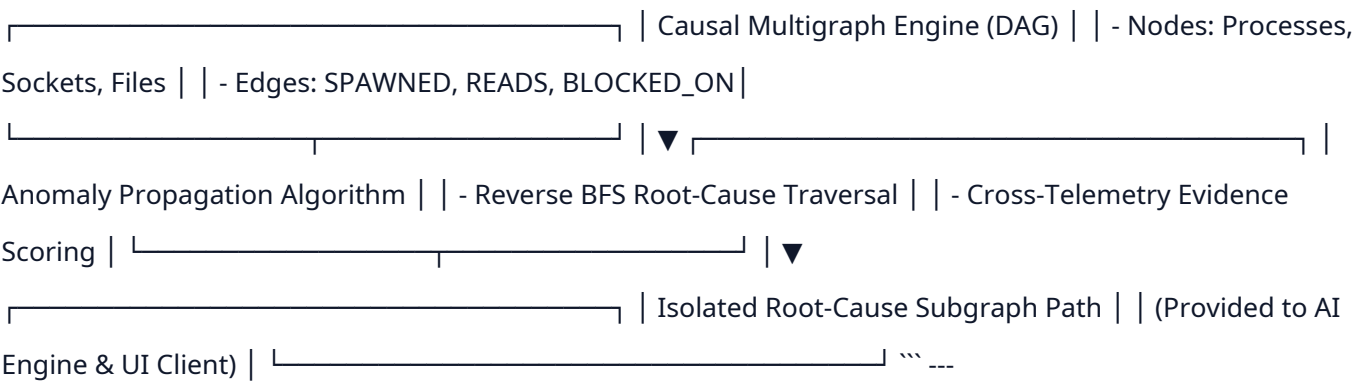
# SysPilot Master Architecture Specification: Volume 4

## Multimodal Causal Graph Engine, IPC Mechanics & Production Safety Controls

---

### 1. Executive Causal Reasoning Architecture

The Causal Reasoning Engine constructs a dynamic topological Directed Acyclic Graph (DAG) linking processes, open file descriptors, network sockets, cgroup containers, and system resources. When a latency spike or error anomaly occurs, the engine traverses this graph to isolate root-cause paths across multiple telemetry sources. `` Telemetry Stream (Kernel Events, Traces, PMUs, Metrics) | ▼



### 2. Causal Graph C++ Class Specifications

#### 2.1 Graph Node & Edge Data Structures (`src/causal/graph\_types.hpp`)

```
``cpp namespace syspilot::causal { enum class NodeType : uint8_t { PROCESS, FILE, SOCKET, CGROUP, DEVICE }; enum class EdgeType : uint8_t { SPAWNED_BY, READS_FROM, WRITES_TO, BLOCKED_ON, CONTENTS_WITH }; struct GraphNode { std::string_view id; // e.g. "pid:4582" or "resource:/dev/sda" NodeType type; pid_t pid{0}; double cpu_pct{0.0}; double read_rate_kb{0.0}; double write_rate_kb{0.0}; bool is_anomalous{false}; std::string_view anomaly_reason; }; struct GraphEdge { std::string_view from_id; std::string_view to_id; EdgeType type; uint64_t latency_ns{0}; std::string_view details; }; class CausalGraph
```

```
{ private: tsl::robin_map nodes_; std::vector edges_; memory::StringArena arena_; public: CausalGraph() =
default; void add_node(GraphNode node) { node.id = arena_.allocate(node.id); nodes_[node.id] = node; } void
add_edge(GraphEdge edge) { edge.from_id = arena_.allocate(edge.from_id); edge.to_id =
arena_.allocate(edge.to_id); edges_.push_back(edge); } // Reverse BFS Root-Cause Traversal Algorithm
std::vector trace_root_cause(std::string_view symptom_id) { std::vector path; tsl::robin_set visited; std::queue
q; q.push(symptom_id); visited.insert(symptom_id); while (!q.empty()) { auto curr = q.front(); q.pop();
path.push_back(curr); for (const auto& edge : edges_) { if (edge.from_id == curr && visited.find(edge.to_id) ==
visited.end()) { visited.insert(edge.to_id); q.push(edge.to_id); } } } return path; } // Calculates composite
anomaly score across correlated metrics double calculate_node_anomaly_score(const GraphNode& node,
double pmu_cache_miss_rate) const noexcept { double w1 = 0.35; // CPU usage weight double w2 = 0.25; //
PMU cache miss weight double w3 = 0.25; // I/O wait rate weight double w4 = 0.15; // Error count weight
double cpu_score = node.cpu_pct / 100.0; double io_score = (node.read_rate_kb + node.write_rate_kb) /
10000.0; return (w1 * cpu_score) + (w2 * pmu_cache_miss_rate) + (w3 * io_score); } void clear() noexcept
{ nodes_.clear(); edges_.clear(); arena_.reset(); } }; } // namespace syspilot::causal "" ---
```

3. Inter-Process Communication (IPC) Specifications

SIOP layers specialized IPC mechanisms according to latency and throughput requirements: ""

	IPC Subsystem	Transport Mechanism	Latency SLA	Target Use Case
	Kernel → Daemon	`BPF_MAP_TYPE_RINGBUF` (mmap)	< 100 nanoseconds	Raw kernel telemetry events
	Daemon → Local	`/dev/shm/syspilot_metrics.ring`	< 50 nanoseconds	Live TUI metric rendering
	Daemon → Client	UNIX Domain Socket (UDS stream)	< 500 microseconds	Interactive CLI queries
	Daemon → Central	gRPC over HTTP/2 + Zstd Stream	< 5 milliseconds	Regional/Central Aggregator
				"" ---

4. The 13 Production Safety & Reliability Principles

1. **CPU Limit (< 1.0% Host CPU Core):** Event-driven eBPF tracepoints and `epoll\_wait` reactor pattern eliminate polling overhead.
2. **Memory Limit (< 64 MiB Resident RAM):** Pre-allocated slab bump arenas (`StringArena`) and fixed pool allocations prevent memory growth.
3. **Zero Hot-Path Dynamic**

Allocations:\*\* All records and string views are populated in-place inside pre-allocated slab pages. 4. \*\*Lockless Multithreading:\*\* Lock-free MPMC queue (`moodycamel::ConcurrentQueue`) and open-addressing maps (`tsl::robin\_map`). 5. \*\*Context Switch Minimization:\*\* Adaptive epoll wakeup timer high-watermark algorithm keeps wakeups < 100 / sec. 6. \*\*Kernel-Userspace Responsibility Separation:\*\* eBPF probes emit raw struct bytes; expensive DWARF unwinding and string formatting occur in userspace. 7. \*\*Backpressure & Drop Mitigation:\*\* Atomic event drop counters in kernel eBPF ring buffer headers notify userspace of lost events. 8. \*\*Monotonic Event Sequence Tracking:\*\* Every event carries a per-CPU monotonic sequence counter to detect event loss gaps. 9. \*\*Adaptive Dynamic Load Shedding:\*\* Automated token bucket rate limiters dynamic throttle telemetry fidelity when host CPU > 80%. 10. \*\*Kernel Instability Immunity:\*\* All eBPF programs pass strict verification by the Linux kernel verifier. Zero kernel panic risk. 11. \*\*Process Crash Isolation:\*\* Independent daemon sandboxing; daemon failure leaves host application execution untouched. 12. \*\*High-Throughput Vectorization:\*\* AVX2/AVX-512 SIMD pipeline supporting > 5,000,000 events / sec / host. 13. \*\*Resource Cgroup Hard Bounds:\*\* Enforced systemd slice caps (`MemoryMax=128M`, `CPUQuota=5%`).